

#10 Divide and Conquer & Binary Search

김성수

4thrunyul@gmail.com

0. 목차



1. Divide and Conquer

- Concept
- Sorting
- Fast Exponentiation

2. Binary Search

- Concept
- Basic Implementation
- `std::binary_search`, `std::lower_bound`, `std::upper_bound`

3. Parametric Search

- Concept

1. Divide and Conquer

1. Divide and Conquer: Concept



Divide and Conquer(분할 정복)

- 큰 문제를 직접 해결하는 대신, 작은 단위로 쪼개어 각각 해결한 뒤 결과를 합치는 알고리즘
 - 1. 분할 : 원래 문제를 동일한 형태의 더 작은 하위 문제들로 나눔
 - 2. 정복 : 하위 문제들을 재귀적으로 해결
 - 3. 결합 : 하위 문제의 해답을 합쳐 원래 문제의 해답을 구함
- 분할 정복을 사용하는 대표적인 알고리즘으로는 병합 정렬, 퀵 정렬, 이분 탐색, 빠른 거듭제곱, 빠른 행렬곱(슈트라센 알고리즘) 등이 있음.

1. Divide and Conquer: Concept



Divide and Conquer(분할 정복)

- 독립 부분 문제
 - 분할된 작은 문제들이 서로 영향을 주지 않고 중복 없이 독립적으로 해결되는 구조
- 자가 유사성
 - 문제를 쪼개도 원래 문제와 동일한 성질과 해결 방식을 유지하는 구조
- 효율적인 결합 구조
 - 작은 문제들의 해를 합쳐 전체 해를 도출하는 과정이 직접 계산하는 것보다 효율적인 구조

1. Divide and Conquer: Sorting

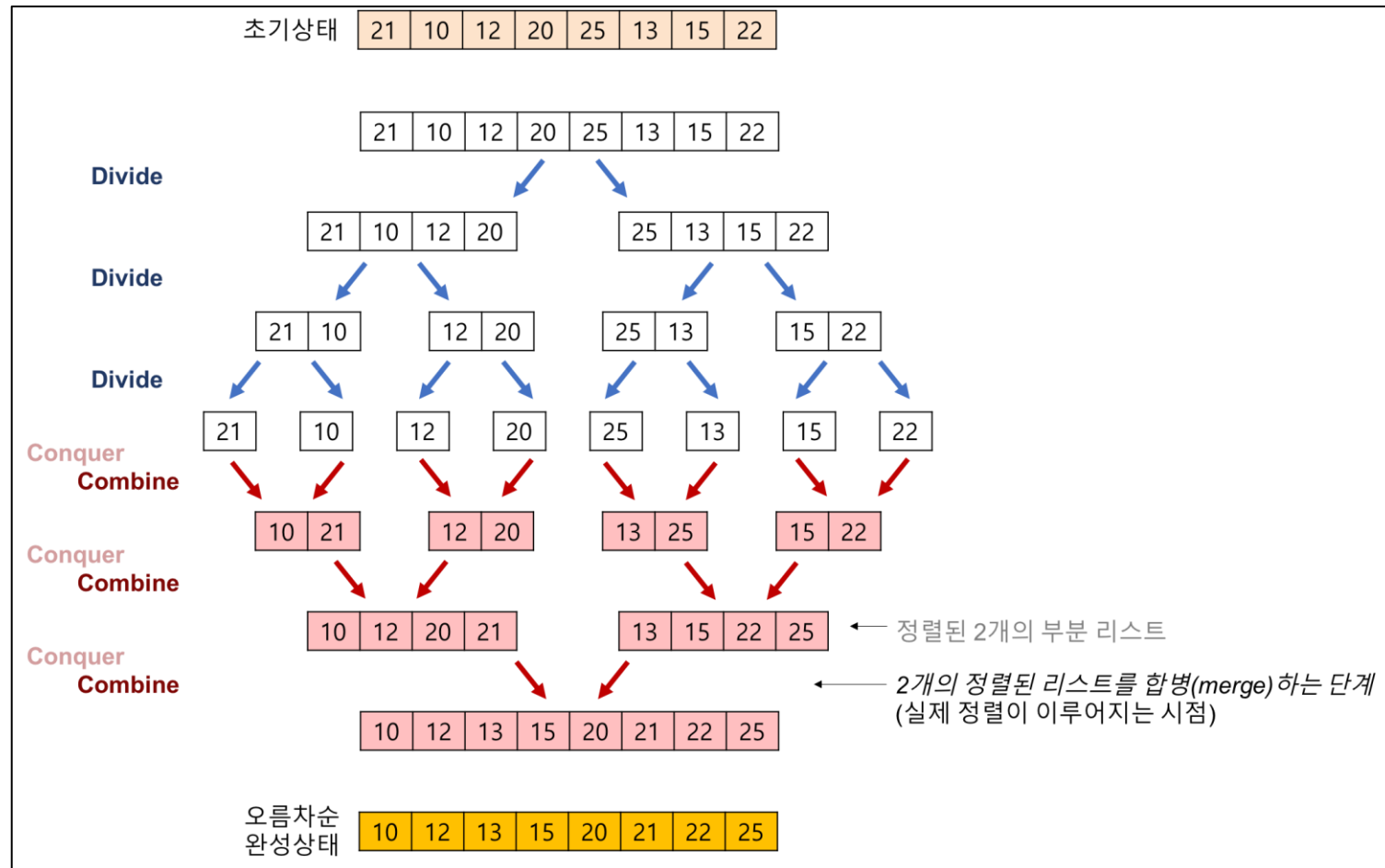


병합 정렬의 분할 정복

- 분할
 - 전체 배열을 절반씩 계속 나눔 하나의 원소만 남는 크기까지 문제를 작은 부분 문제로 분할
- 정복
 - 원소가 하나인 배열은 이미 정렬된 상태로 보고, 가장 작은 부분 문제부터 해결하면서 정렬된 부분 배열을 구성
- 결합
 - 정렬된 두 부분 배열을 비교하며 작은 값부터 순서대로 병합하고, 이 과정을 반복하여 최종적으로 전체 배열을 정렬된 하나의 배열로 완성

1. Divide and Conquer: Sorting

병합 정렬의 분할 정복



1. Divide and Conquer: Sorting

문제

- [3519 Tutorial : 합병\(병합\)정렬](#)
- 코드

```
1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  int n;
7  vector<int> v;
8
9  void merge(int left, int right) {
10     if(left >= right) return; // 기저 조건
11     else {
12         int mid = (left + right) / 2;
13
14         // 분할 정복
15         merge(left, mid);
16         merge(mid + 1, right);
17
18         // 결합
19         int i = left;
20         int j = mid + 1;
21         vector<int> temp(right - left + 1);
22
23         for (int k = 0; k < temp.size(); k++) {
24             if (i <= mid && (j > right || v[i] <= v[j])) temp[k] = v[i++];
25             else temp[k] = v[j++];
26         }
27     }
28 }
```

```
28     // 복사
29     for (int t = 0; t < temp.size(); t++) {
30         v[left + t] = temp[t];
31     }
32
33     for (int i = 0; i < n; i++) {
34         cout << v[i] << " ";
35     }
36
37     cout << "\n";
38 }
39
40
41 int main() {
42     ios_base::sync_with_stdio(false);
43     cin.tie(NULL);
44
45     int num;
46
47     cin >> n;
48
49     for (int i = 0; i < n; i++) {
50         cin >> num;
51
52         v.push_back(num);
53     }
54
55     merge(0, n - 1);
56
57     return 0;
58 }
```

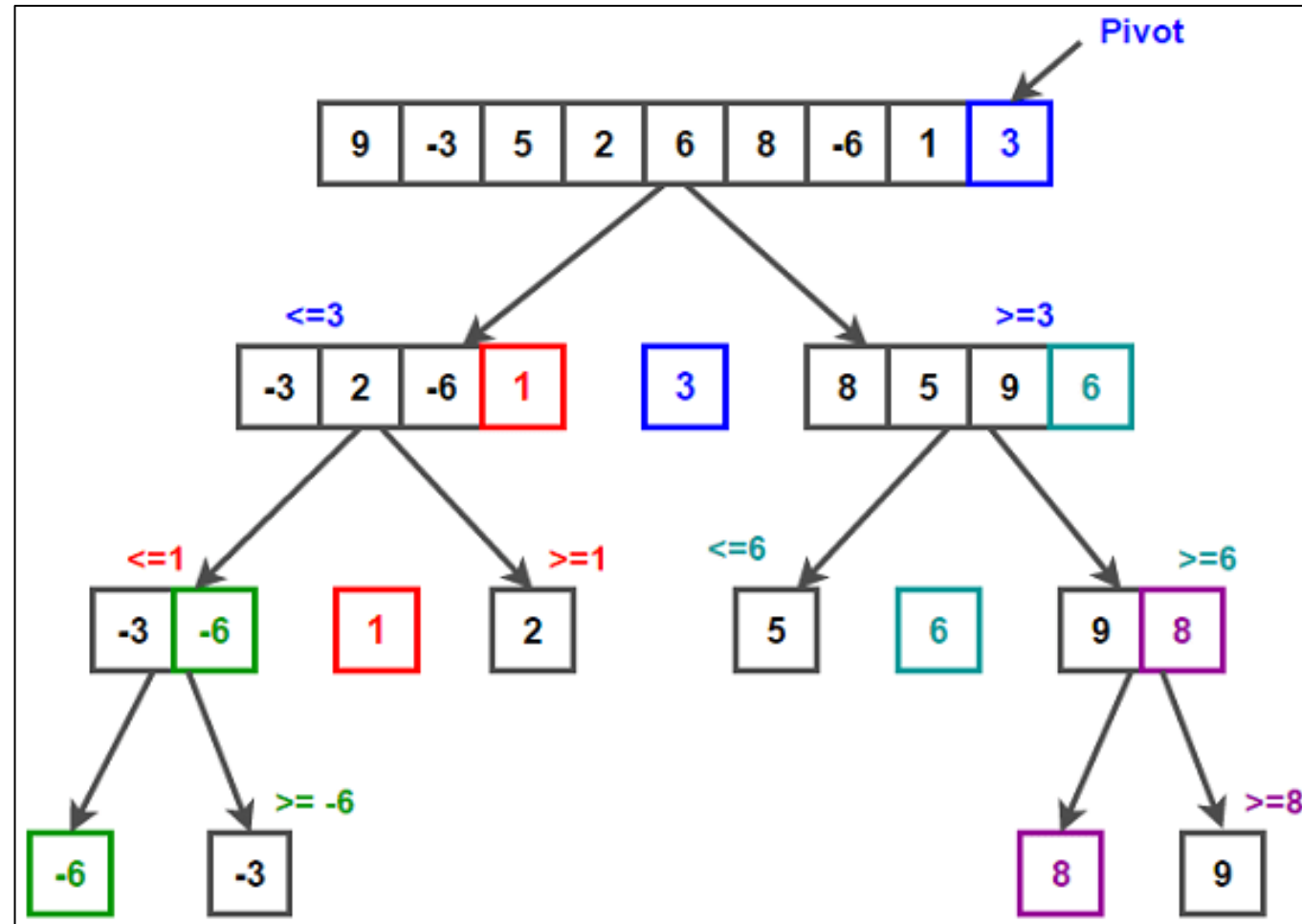
1. Divide and Conquer: Sorting

병합 정렬의 분할 정복

- Base condition
 - 정렬할 배열에서 원소가 하나 이하인 구간은 이미 정렬된 것으로 보고 더 이상 분할하지 않음
- Select Pivot
 - 현재 정렬할 구간에서 하나의 원소를 피벗으로 선택하여 분할의 기준으로 사용
- 분할
 - 피벗보다 작은 값들은 왼쪽 부분에, 큰 값들은 오른쪽 부분에 위치하도록 배열을 분할
 - 이후 피벗의 위치를 결정
- 정복
 - 피벗을 제외한 왼쪽 부분 배열과 오른쪽 부분 배열에 동일한 퀵소트 과정을 재귀적으로 적용
 - 부분 배열의 크기가 하나 이하가 될 때까지 분할과 정복을 반복하여 전체 배열의 정렬을 완성

1. Divide and Conquer: Sorting

퀵정렬의 분할 정복



1. Divide and Conquer: Sorting



문제

- [3518 Tutorial : 퀵정렬](#)
 - 코드

1. Divide and Conquer: Fast Power



Fast Exponentiation

- 정수 x, n 에 대해 x^n 를 구하려고 할 때
- 일반적인 방법
 - x 를 n 번 반복해서 구할 수 있음.
 - 구현하기 쉽지만 n 이 커질수록 곱셈 횟수도 비례하여 증가하므로 $O(n)$ 의 시간 복잡도를 가짐
- 분할정복 적용
 - 거듭제곱의 성질을 이용하면 x^n 을 더 작은 지수의 거듭제곱 문제로 나눌 수 있음.
 - 지수 n 을 절반씩 줄여가며 계산하고, 구한 값을 재사용하여 전체 결과를 구함.
- 효율성
 - 매 단계마다 지수가 절반으로 감소하므로 필요한 연산 횟수가 $O(n)$ 에서 $O(\log n)$ 으로 줄어듦
 - 때문에 큰 지수도 효율적으로 계산할 수 있음

1. Divide and Conquer: Fast Power



Fast Exponentiation

- n 이 짝수인 경우
 - $x^n = x^{(n/2)} * x^{(n/2)}$
 - $x^{(n/2)}$ 을 한 번만 계산한 뒤 그 값을 제공하여 결과를 구함
- n 이 홀수인 경우
 - $x^n = x * x^{((n-1)/2)} * x^{((n-1)/2)}$
 - $x^{((n-1)/2)}$ 을 한 번 계산한 뒤 그 값을 제공하고 x 를 한 번 더 곱하여 결과를 구함

1. Divide and Conquer: Fast Power



문제

- 1092 제공수 출력
- 코드

```
1  #include <iostream>
2
3  using namespace std;
4
5  ✓ long long pow(long long a, long long b) {
6      if(b == 0) return 1;
7
8      long long temp = pow(a, b / 2);
9
10     if(b % 2 == 0) return (temp * temp) % 20091024;
11     else return (((temp * temp) % 20091024) * (a % 20091024)) % 20091024;
12 }
13
14 ✓ int main() {
15     long long x, y, result;
16
17     cin >> x >> y;
18
19     result = pow(x, y);
20
21     cout << result;
22
23     return 0;
24 }
```

1. Divide and Conquer: 실습



실습

- 1335 색종이 만들기(영역 구분)
 - 코드
- 3560 색종이 만들기2(4진 트리)
 - 코드
- 4414 이진 복원

2. Binary Search

2. Binary Search: Concept



Up-Down Game

- 제가 1~100 중에 한 가지의 수를 떠올렸습니다.

2. Binary Search: Concept



Up-Down Game

- 제가 떠올린 수는... 15였습니다!
- Q1 : 50 or 51
 - A1 : Down
- Q2 : 25 or 26
 - A2 : Down
- Q3 : 12 or 13
 - A3 : Up
- Q4 : 19 or 20
 - A4 : Down...

2. Binary Search: Concept

선형 탐색 vs 이분 탐색

- 선형 탐색 : $O(n)$
- 이분 탐색 : $O(\log n)$
- ex) $n = 100,000,000$ (1억)
- 선형 탐색 : 최악의 경우 1억 번 연산
- 이분 탐색 : 최악의 경우 27번 연산
- $n = 100,000,000$ 인 상태에서 Q번의 쿼리가 입력되어 m의 개수를 출력하라고 하면 어떻게 될까?
- 선형 탐색 : $O(Q * n)$
- 이분 탐색 : $O(Q * \log n)$

이분 탐색의 전제 조건

- 탐색을 하고자 하는 데이터가 **오름차순으로 정렬**이 되어있어야 함
- 내림차순도 가능하긴 하지만, 따로 구현해야 할 것들이 많음에 따라 해당 PPT에서는 다루지 않음

2. Binary Search: Basic Implementation



```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4
5  using namespace std;
6
7  int main() {
8      int target = 3;
9      int cnt = 0;
10     vector<int> v = {1, 2, 3, 4, 5, 6, 7};
11
12     int low = 0;
13     int high = v.size() - 1;
14     int result = -1;
15
16     while (low <= high) {
17         ++cnt;
18
19         int mid = (low + high) / 2;
20
21         if (v[mid] == target) {
22             result = mid;
23             break;
24         }
25         else if (v[mid] > target) high = mid - 1;
26         else low = mid + 1;
27     }
28
29     cout << "index: " << result;
30     cout << "\ncalc: " << cnt;
31
32     return 0;
33 }
```

while (low <= high)

- 탐색 범위에 원소가 남아 있는 동안 반복

int mid = (low + high) / 2;

- 현재 탐색 범위의 가운데 인덱스를 계산하며, 정수 나눗셈이므로 소수점은 버림

else if (v[mid] > target) high = mid - 1;

- 가운데 원소가 찾을 값보다 크면, 가운데 원소와 오른쪽을 제외하고 왼쪽에서 탐색

else low = mid + 1;

- 가운데 원소가 찾을 값보다 작으면, 가운데 원소와 왼쪽을 제외하고 오른쪽에서 탐색

```
Input:                                     Copy
Expected Output:                           Copy
index: 2
calc: 3
```

2. Binary Search: Basic Implementation



BINARY SEARCH / while 반복문 실행 과정

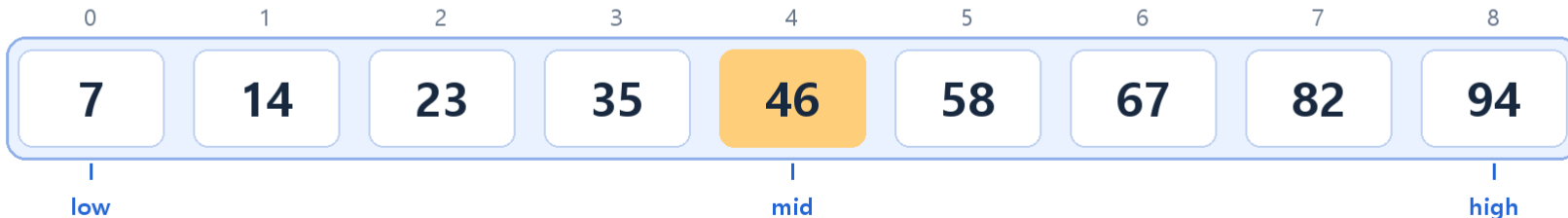
1회차

현재 탐색 범위: 인덱스 0 ~ 8

target = 35

현재 탐색 범위

아래 배열은 mid 계산 직후, if 비교 전 상태입니다.



파란 테두리: 탐색할 원소 회색: 제외된 원소 강조된 칸: 이번에 비교할 v[mid]

low

0

high

8

mid

4

v[mid]

46

cnt

1

while (0 <= 8) → True

mid = (0 + 8) / 2 = 4 (정수 나눗셈)

if (v[mid] == target)

46 == 35 → False

else if (v[mid] > target)

46 > 35 → True

실행 high = mid - 1 = 3;

다음 회차: low = 0, high = 3 | 탐색할 원소: [7, 14, 23, 35]

인덱스는 0부터 시작합니다. low-high는 경계 인덱스, mid는 가운데 인덱스입니다.

2. Binary Search: Basic Implementation



BINARY SEARCH / while 반복문 실행 과정

2회차

현재 탐색 범위: 인덱스 0 ~ 3

target = 35

현재 탐색 범위

아래 배열은 mid 계산 직후, if 비교 전 상태입니다.



파란 테두리: 탐색할 원소 회색: 제외된 원소 강조된 칸: 이번에 비교할 v[mid]

low

0

high

3

mid

1

v[mid]

14

cnt

2

while (0 <= 3) → True

mid = (0 + 3) / 2 = 1 (정수 나눗셈)

if (v[mid] == target)

14 == 35 → False

else if (v[mid] > target)

14 > 35 → False

실행

else 실행 → low = mid + 1 = 2;

다음 회차: low = 2, high = 3 | 탐색할 원소: [23, 35]

인덱스는 0부터 시작합니다. low-high는 경계 인덱스, mid는 가운데 인덱스입니다.

2. Binary Search: Basic Implementation



BINARY SEARCH / while 반복문 실행 과정

3회차

현재 탐색 범위: 인덱스 2 ~ 3

target = 35

현재 탐색 범위

아래 배열은 mid 계산 직후, if 비교 전 상태입니다.



파란 테두리: 탐색할 원소 회색: 제외된 원소 강조된 칸: 이번에 비교할 v[mid]

low

2

high

3

mid

2

v[mid]

23

cnt

3

while (2 <= 3) → True

mid = (2 + 3) / 2 = 2 (정수 나눗셈)

if (v[mid] == target)

23 == 35 → False

else if (v[mid] > target)

23 > 35 → False

실행

else 실행 → low = mid + 1 = 3;

다음 회차: low = 3, high = 3 | 탐색할 원소: [35]

인덱스는 0부터 시작합니다. low-high는 경계 인덱스, mid는 가운데 인덱스입니다.

2. Binary Search: Basic Implementation



BINARY SEARCH / while 반복문 실행 과정

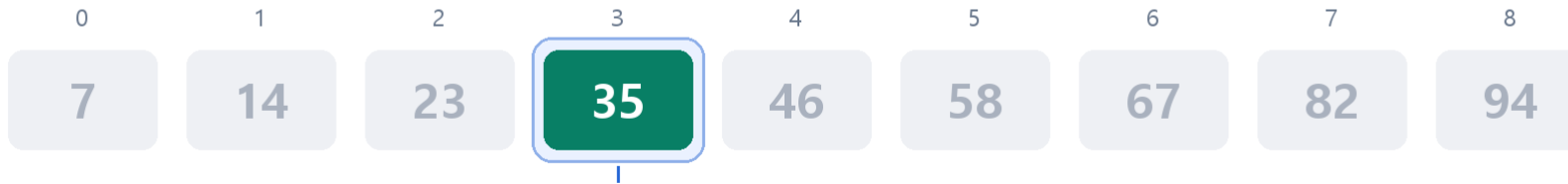
4회차

현재 탐색 범위: 인덱스 3 ~ 3

target = 35

현재 탐색 범위

아래 배열은 mid 계산 직후, if 비교 전 상태입니다.



low · mid · high

파란 테두리: 탐색할 원소 회색: 제외된 원소 강조된 칸: 이번에 비교할 v[mid]

low	3	high	3	mid	3	v[mid]	35	cnt	4
-----	---	------	---	-----	---	--------	----	-----	---

while (3 <= 3) → True

mid = (3 + 3) / 2 = 3 (정수 나눗셈)

if (v[mid] == target)

35 == 35 → True

else if (v[mid] > target)

평가하지 않음 (위의 if에서 break)

실행 result = mid = 3; break;

탐색 성공 · index: 3, calc: 4 · while 조건이 False가 아니라 break로 종료

인덱스는 0부터 시작합니다. low-high는 경계 인덱스, mid는 가운데 인덱스입니다.

2. Binary Search: Basic Implementation



문제

- [3517 Tutorial: 이진탐색](#)
 - [코드](#)

2. Binary Search: std

```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4
5  using namespace std;
6
7  int main() {
8      int n = 5;
9      int m = 10;
10     vector<int> v = {5, 1, 4, 3, 2, 7, 6};
11
12     sort(v.begin(), v.end());
13
14     cout << binary_search(v.begin(), v.end(), n);
15     cout << "\n";
16     cout << binary_search(v.begin(), v.end(), m);
17
18     return 0;
19 }
```

Input:

Copy

Expected Output:

Copy

1
0

binary_search(시작 위치, 끝 위치, 찾을 값)

<algorithm> | $O(\log n)$

- 지정한 범위에 속에 찾으려는 값 존재 여부를 bool 타입으로 반환
- 값이 존재한다면 true(1), 존재하지 않는다면 false(0) 반환
- 벡터의 경우 반복자(iterator)를, 배열의 경우 포인터를 인자로 넣어줘야 함

2. Binary Search: std

```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4
5  using namespace std;
6
7  int main() {
8      int n = 4;
9      int m = 10;
10     vector<int> v = {1, 2, 3, 4, 5, 6, 7};
11
12     // 4 이상의 값이 처음 나오는 위치
13     // (값: 4, 인덱스: 3)
14     cout << lower_bound(v.begin(), v.end(), n) - v.begin();
15     cout << "\n";
16
17     // 10 이상의 값이 처음 나오는 위치
18     // (없으므로 v.end()인 인덱스: 7)
19     cout << lower_bound(v.begin(), v.end(), m) - v.begin();
20     cout << "\n";
21
22     // 4를 초과하는 값이 처음 나오는 위치
23     // (값: 5, 인덱스: 4)
24     cout << upper_bound(v.begin(), v.end(), n) - v.begin();
25     cout << "\n";
26
27     // 10을 초과하는 값이 처음 나오는 위치
28     // (없으므로 v.end()인 인덱스: 7)
29     cout << upper_bound(v.begin(), v.end(), m) - v.begin();
30
31     return 0;
32 }
```

lower_bound(시작 위치, 끝 위치, 찾을 값)

<algorithm> | $O(\log n)$

- 지정한 범위 내에서 찾을 값 이상의 값이 처음으로 나타나는 위치를 반환
- 조건에 맞는 원소가 없으면 끝 위치를(v.end()) 반환
- 반환형은 반복자(iterator) 또는 포인터이므로, 인덱스를 구하려면 시작 위치를 빼줘야 함

upper_bound(시작 위치, 끝 위치, 찾을 값)

<algorithm> | $O(\log n)$

- 지정한 범위 내에서 찾을 값 초과하는 값이 처음으로 나타나는 위치를 반환
- 조건에 맞는 원소가 없으면 끝 위치를(v.end()) 반환
- 두 함수의 반환 위치 차이(upper_bound - lower_bound)를 통해 vector나 array이 포함하는 특정 원소의 개수를 빠르게 구할 수 있음

Input:	Copy
Expected Output:	Copy
3	
7	
4	
7	

2. Binary Search: std



문제

- [8069 Tutorial: 이진탐색\(라이브러리 사용법\)](#)
 - [코드](#)

2. Binary Search: 실습



실습

- [7009 도둑 잡아라](#)
 - [코드](#)
- [8653 뽕족산](#)
 - [코드](#)

3. Parametric Search

3. Parametric Search: Concept

매개변수 탐색(Parametric Search)

- 문제에서 요구하는 최적화 문제(최댓값이나 최솟값을 찾는 문제)를 결정 문제로 바꾸어 풀이하는 기법
- ex) 16층인 건물이 있다. 계란을 던졌을 때 계란이 깨지지 않는 최대 층을 구하세요.
 - Case1 : 1~5층 안깨짐, 6~16층 깨짐
 - A. 8층 : 깨짐
 - B. 4층 : 안깨짐
 - C. 6층 : 깨짐
 - D. 5층 : 안깨짐
 - 경계값을 찾아냈기 때문에 5층이 최대층인 것을 알 수 있음
 - Case2 : 1~10층 안깨짐, 11~16층 깨짐
 - A. 8층 : 안깨짐
 - B. 12층 : 깨짐
 - C. 10층 : 안깨짐
 - D. 11층 : 깨짐
 - 경계값을 찾아냈기 때문에 10층이 최대층인 것을 알 수 있음

3. Parametric Search: Concept



매개변수 탐색의 조건

- 1. 가능한 정답들이 일정한 순서를 가지는 탐색 구간을 형성해야 함.
- 2. 단조성이 있어야 함.
 - 특정 지점을 기준으로 조건의 참/거짓이 한 방향으로만 변해야 함
 - 앞서 건물에서 계란을 떨어뜨리는 문제에 성립할 수 있었던 이유
 - n 층에서 깨짐 $\rightarrow n$ 이상의 층에서도 계란이 깨지구나
 - n 층에서 안깨짐 $\rightarrow n$ 이하의 층에서도 계란이 안깨지구나

3. Parametric Search: Concept



문제

- [8654 Tutorial: 이진탐색 응용](#)
 - [코드](#)

3. Parametric Search: 실습



실습

- [5170 나무꾼 미르코](#)
 - [코드](#)
- [2581 예산](#)
 - [코드](#)
- [5461 숫자구슬\(medium\)](#)
 - [코드](#)